Perform the following operations using Python on Heart Diseases data sets

- Data cleaning
- Data integration
- Data transformation
- Error correcting
- Data model building

In [2]:
```
# import pandas library
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score,confusion_matrix
from sklearn.linear_model import LogisticRegression
import seaborn as sns
import matplotlib.pyplot as plt
```

In [3]:
```
# Reading csv file
df = pd.read_csv("Heart.csv")
df.head()
```

Out[3]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	c
0	52	1	0	125	212	0	1	168	0	1.0	2	
1	53	1	0	140	203	1	0	155	1	3.1	0	
2	70	1	0	145	174	0	1	125	1	2.6	0	
3	61	1	0	148	203	0	1	161	0	0.0	2	
4	62	0	0	138	294	1	1	106	0	1.9	1	

Data Cleaning

In [4]:
```
df = df.drop_duplicates()
```

In [5]:
```
# Count ,min,max ,etc of each column
df.describe()
```

Out[5]:

	age	sex	cp	trestbps	chol	fbs	3(
count	302.00000	302.000000	302.000000	302.000000	302.000000	302.000000	
mean	54.42053	0.682119	0.963576	131.602649	246.500000	0.149007	
std	9.04797	0.466426	1.032044	17.563394	51.753489	0.356686	
min	29.00000	0.000000	0.000000	94.000000	126.000000	0.000000	
25%	48.00000	0.000000	0.000000	120.000000	211.000000	0.000000	
50%	55.50000	1.000000	1.000000	130.000000	240.500000	0.000000	
75%	61.00000	1.000000	2.000000	140.000000	274.750000	0.000000	
max	77.00000	1.000000	3.000000	200.000000	564.000000	1.000000	

In [6]:

```
# Information about each column data
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 302 entries, 0 to 878
Data columns (total 14 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   age       302 non-null    int64
 1   sex       302 non-null    int64
 2   cp        302 non-null    int64
 3   trestbps  302 non-null    int64
 4   chol      302 non-null    int64
 5   fbs       302 non-null    int64
 6   restecg   302 non-null    int64
 7   thalach   302 non-null    int64
 8   exang     302 non-null    int64
 9   oldpeak   302 non-null    float64
 10  slope     302 non-null    int64
 11  ca        302 non-null    int64
 12  thal      302 non-null    int64
 13  target    302 non-null    int64
dtypes: float64(1), int64(13)
memory usage: 35.4 KB
```

In [7]:

```
#Finding null values in each column
df.isna().sum()
```

Out[7]:
```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

Data Integration

In [8]:
```
df.head()
```

Out[8]:

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	c
0	52	1	0	125	212	0	1	168	0	1.0	2	
1	53	1	0	140	203	1	0	155	1	3.1	0	
2	70	1	0	145	174	0	1	125	1	2.6	0	
3	61	1	0	148	203	0	1	161	0	0.0	2	
4	62	0	0	138	294	1	1	106	0	1.9	1	

In [9]:
```
df.fbs.unique()
```

Out[9]:
```
array([0, 1])
```

In [10]:
```
subSet1 = df[['age','cp','chol','thalach']]
```

In [11]:
```
subSet2 = df[['exang','slope','target']]
```

In [12]:
```
merged_df = subSet1.merge(right=subSet2,how='cross')
merged_df.head()
```

Out[12]:

	age	cp	chol	thalach	exang	slope	target
0	52	0	212	168	0	2	0
1	52	0	212	168	1	0	0
2	52	0	212	168	1	0	0
3	52	0	212	168	0	2	0
4	52	0	212	168	0	1	0

Error Correcting

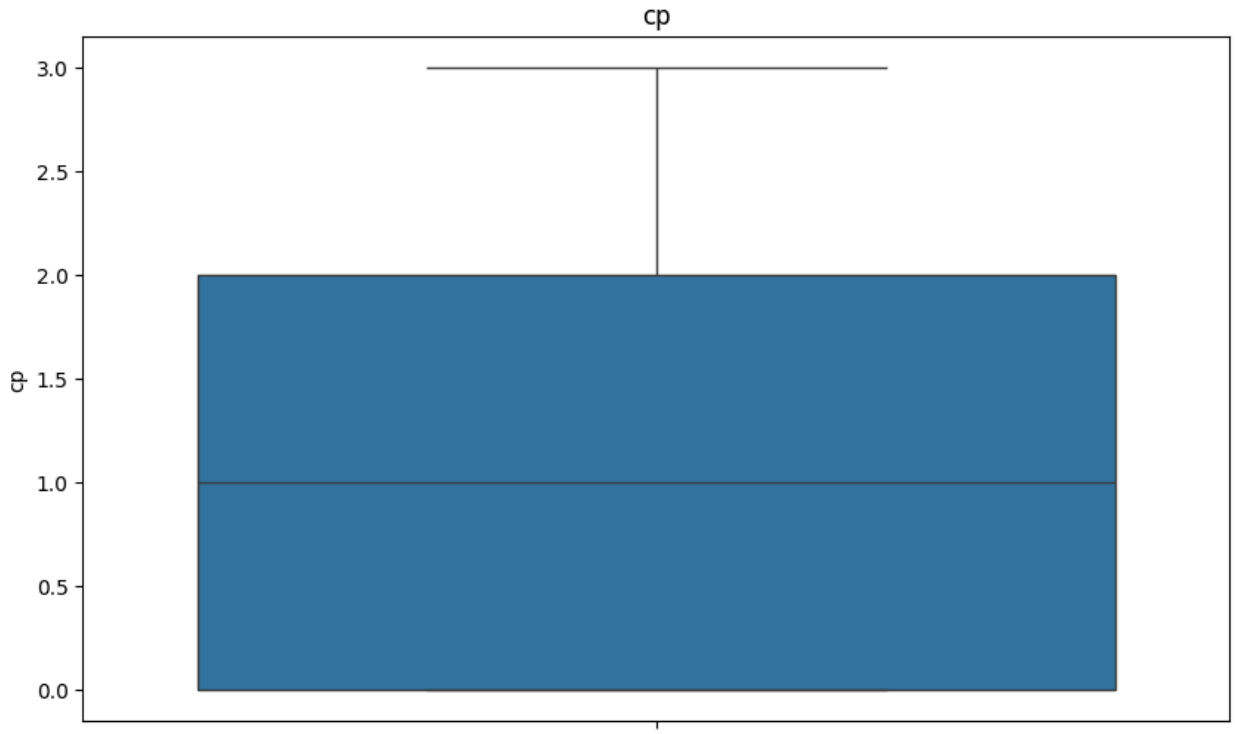
In [13]:
```
df.columns
```

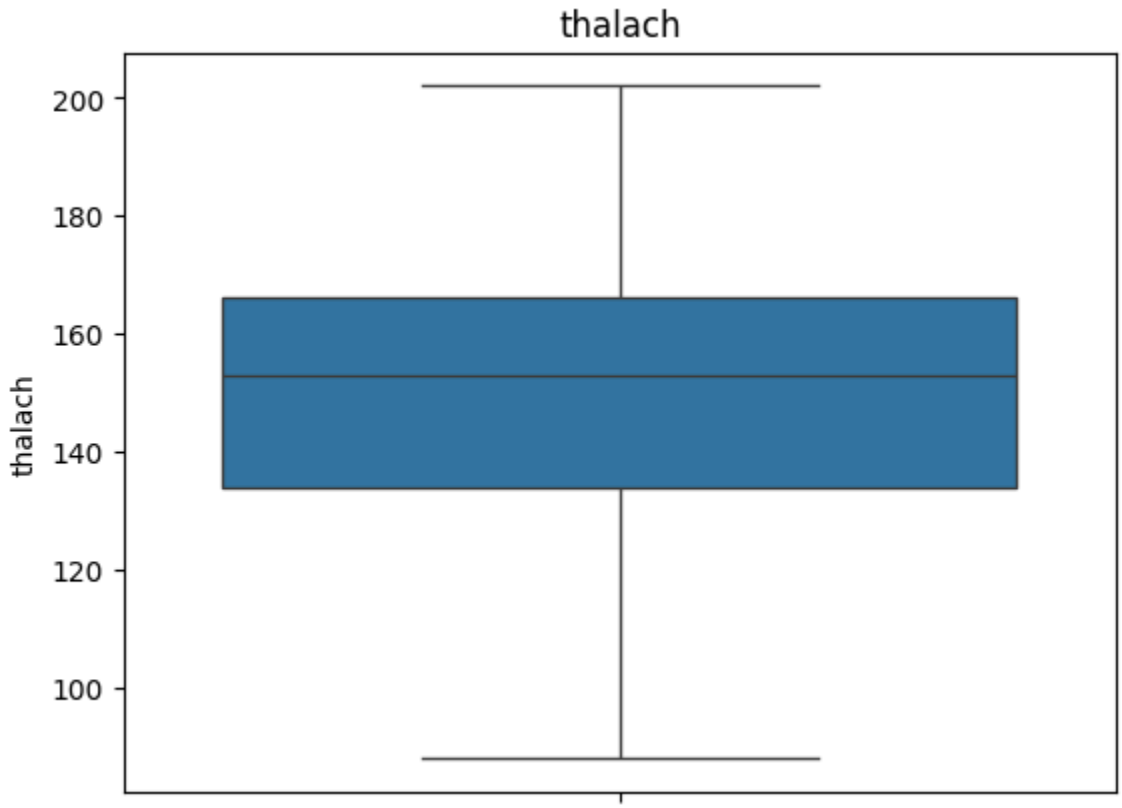
Out[13]:
```
Index(['age', 'sex', 'cp', 'trestbps', 'chol', 'fbs', 'restecg', 'thalach',
       'exang', 'oldpeak', 'slope', 'ca', 'thal', 'target'],
      dtype='object')
```

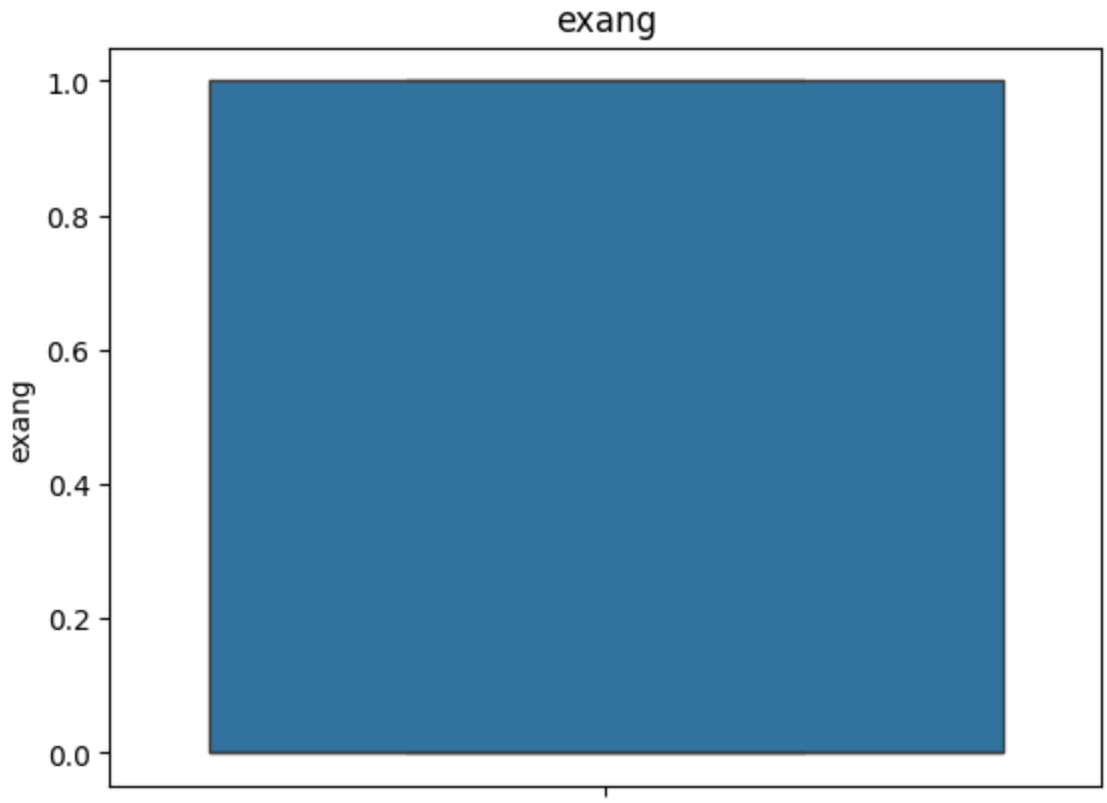
In [14]:
```
def remove_outliers(column):
    Q1 = column.quantile(0.25)
    Q3 = column.quantile(0.75)
    IQR = Q3 - Q1
    threshold = 1.5 * IQR
    outlier_mask = (column < Q1 - threshold) | (column > Q3 + threshold)
    return column[~outlier_mask]
```

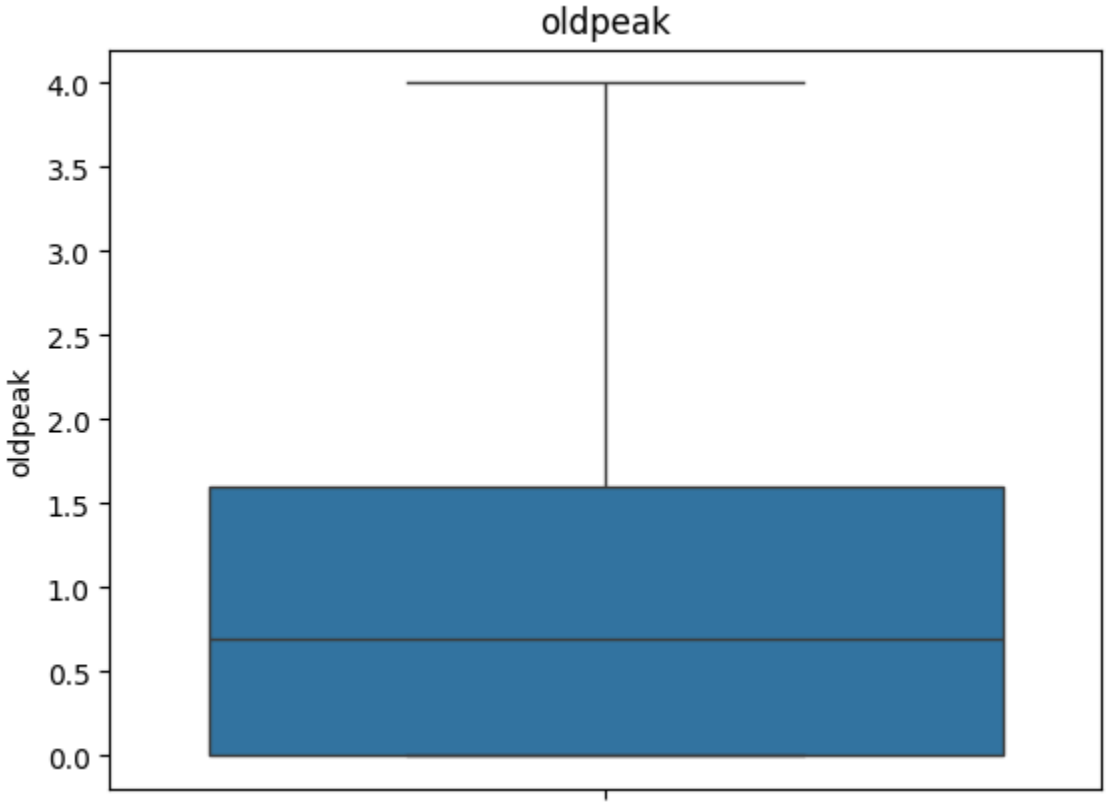
In [15]:
```
# Remove outliers for each column using a loop
col_name = ['cp','thalach','exang','oldpeak','slope','ca']
for col in col_name:
    df[col] = remove_outliers(df[col])
```

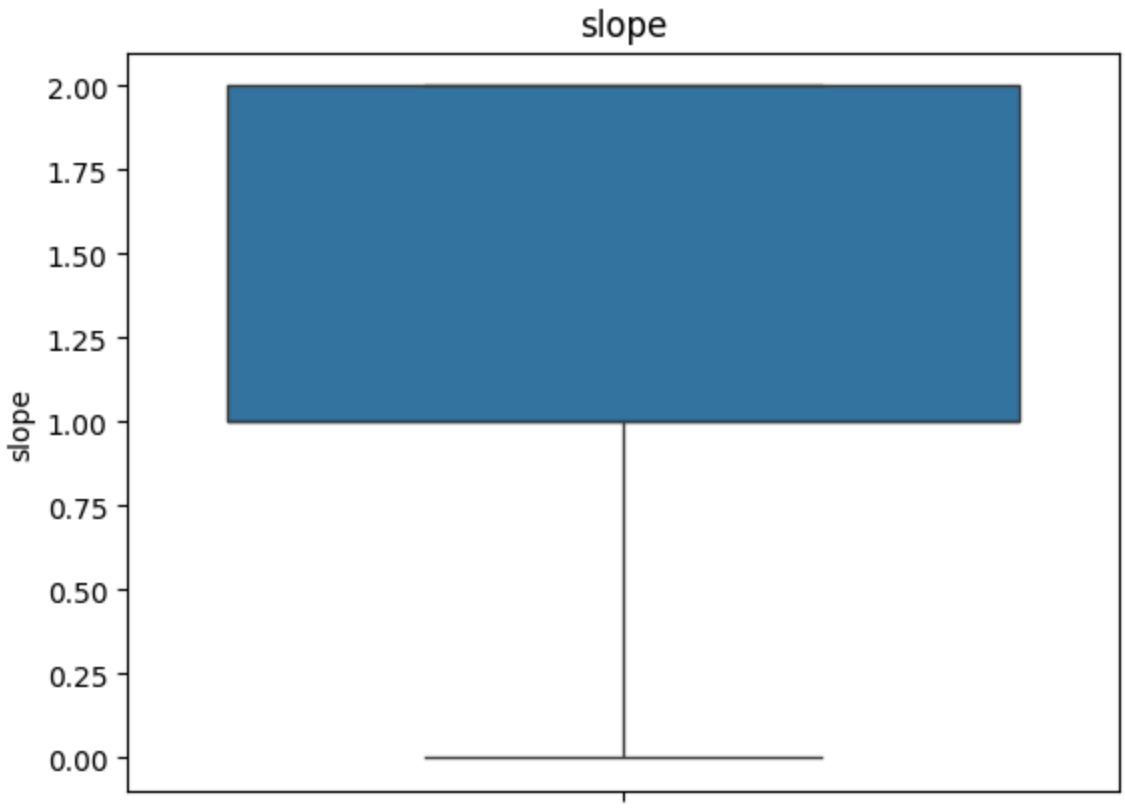
In [16]:
```
plt.figure(figsize=(10, 6))  # Adjust the figure size if needed

for col in col_name:
    sns.boxplot(data=df[col])
    plt.title(col)
    plt.show()
```

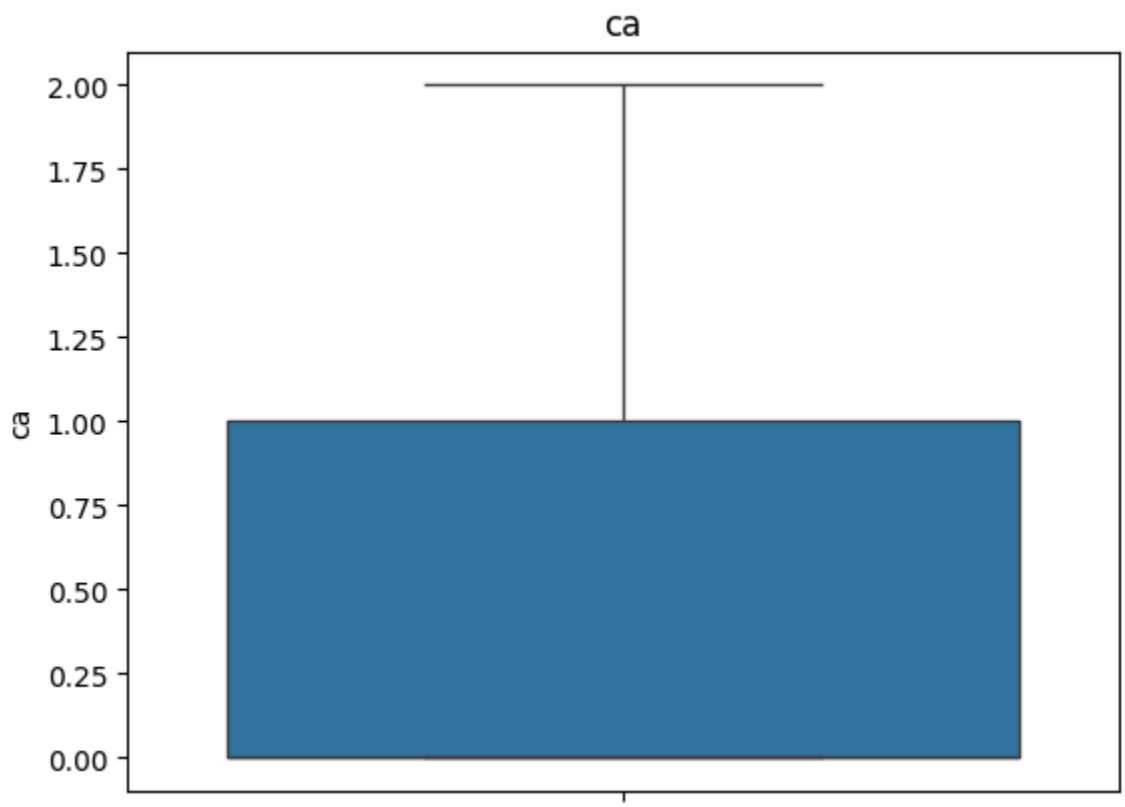

cp
3.0
2.5
2.0
1.5
1.0
0.5
0.0
cp


thalach
200
180
160
140
120
100
thalach


exang
exang
1.0
0.8
0.6
0.4
0.2
0.0


oldpeak
oldpeak
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0





In [17]:
```
df = df.dropna()
```

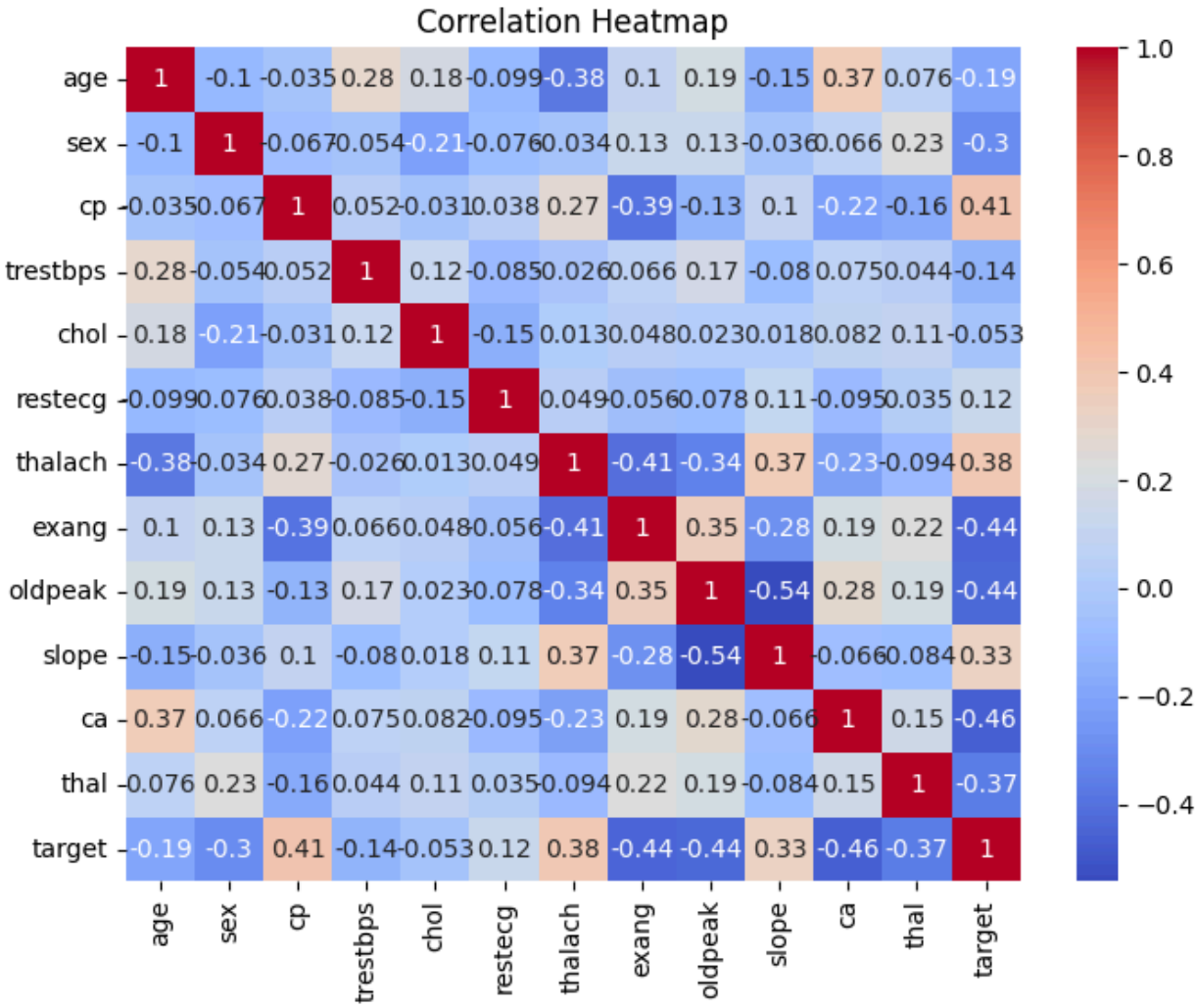
In [18]:
```
df.isna().sum()
```

Out[18]:
```
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

In [19]:
```
df = df.drop('fbs',axis=1)
```

In [21]:
```
# Compute correlations between features and target
correlations = df.corr()['target'].drop('target')

# Print correlations
print("Correlation with the Target:")
print(correlations)
print()

# Plot correlation heatmap
plt.figure(figsize=(8, 6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

```
Correlation with the Target:
age        -0.193798
sex        -0.303271
cp          0.410807
trestbps   -0.135238
chol       -0.052796
restecg     0.122071
thalach     0.384609
exang      -0.444401
oldpeak    -0.437895
slope       0.329432
ca         -0.460816
thal       -0.366390
Name: target, dtype: float64
```



In [22]:
```
# df.isna().sum()
```

Data Split

In [23]:
```
# splitting data using train test split
x = df[['cp','thalach','exang','oldpeak','slope','ca']]
y = df.target
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=

x_train.shape,x_test.shape,y_train.shape,y_test.shape
```

Out[23]:
```
((220, 6), (55, 6), (220,), (55,))
```

Data transformation

In [24]:
```
from sklearn.preprocessing import StandardScaler
```

In [25]:
```
scaler = StandardScaler()
```

In [26]:
```
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)
```

Data model building

In [27]:
```
y_train= np.array(y_train).reshape(-1, 1)
y_test= np.array(y_test).reshape(-1, 1)
```

In [28]:
```
y_train.shape
```

Out[28]:
```
(220, 1)
```

In [29]:
```
model = LogisticRegression()
model.fit(x_train_scaled, y_train)

# Make predictions on the test set
y_pred = model.predict(x_test_scaled)

# Evaluate the model's accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

```
Accuracy: 0.8181818181818182
```

```
C:\Users\HP\AppData\Local\Programs\Python\Python314\Lib\site-packages\sklearn\u
tils\validation.py:1352: DataConversionWarning: A column-vector y was passed wh
en a 1d array was expected. Please change the shape of y to (n_samples, ), for
example using ravel().
  y = column_or_1d(y, warn=True)
```

In [30]:
```
#Classification model using Decision Tree
from sklearn.tree import DecisionTreeClassifier
tc=DecisionTreeClassifier(criterion='entropy')
tc.fit(x_train_scaled,y_train)
y_pred=tc.predict(x_test_scaled)

print("Training Accuracy Score :",accuracy_score(y_pred,y_test))
print("Training Confusion Matrix  :",confusion_matrix(y_pred,y_test))
```

```
Training Accuracy Score : 0.8181818181818182
Training Confusion Matrix  : [[16  6]
 [ 4 29]]
```